

CGI Capacity Analyzer for Director Workload and RFP Assignment

Freya Kan, Qian Yang, Junxian Lin, Yixiao Jing, Jai
Master of Data Science in Computational Linguistics
University of British Columbia

Abstract

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. We develop a Capacity Analyzer that combines historical CRM opportunity data with proposal and RFP text to support director capacity estimation and RFP assignment. The system is designed as a transparent decision-support tool rather than a replacement for business judgment. Week 1 established the reproducible opportunity data foundation and module skeletons. In Week 2, we extended this foundation into a first integrated prototype by refining the cleaned opportunity dataset, validating scoring fields, implementing an initial director-level capacity scoring workflow, connecting capacity outputs to the Streamlit dashboard, and advancing the RFP pipeline toward sample preprocessing, chunking, and retrieval-style output.

1 Introduction

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. Today, capacity may be tracked manually or inconsistently, making it difficult to see which directors are available, nearing full capacity, or well matched to a new opportunity.

This project builds a CGI Capacity Analyzer to support two related business decisions. First, the Director Capacity Dashboard summarizes each director's current workload, historical baseline, relative load, capacity score, and capacity label. Second, the RFP Assignment Tool estimates RFP effort, retrieves similar historical proposal examples, and recommends suitable directors based on experience, available capacity, service-solution fit, and risk.

The system is not intended to replace stakeholder judgment. Instead, it provides a transparent and reproducible data-based starting point that CGI stake-

holders can review, validate, and adjust. This design choice is important because the project does not currently have manually labeled ground truth for director capacity, RFP effort, or correct director assignment.

2 Data

2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two opportunity exports are referred to as opps1 and opps2. The proposal data contains historical proposal and response document pairs.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control.

2.2 Opportunity Data

Initial EDA indicates that opps1 and opps2 are different views of largely overlapping CRM opportunity data. We use opps2 as the primary base table because it contains the richer CRM schema and has one unique opportunity_id per row in the current export. We use opps1 as a secondary source: selected fields from opps1 are collapsed to one row per opportunity_id and merged into opps2, while opps1-exclusive opportunities are appended to the final opportunity_df. Fields that appear under different names across the two exports, especially revenue-related fields, require normalization and validation before downstream scoring and are handled through the Week 2 validation and fallback process.

This distinction is important because the downstream capacity model is expected to operate at the opportunity level. During the Week 1 merge implementation, we found that opps1 contains duplicate opportunity_id values. These duplicates

Data source	Approx. size	Main contents	Role in project
opps1 CRM export	8,435 rows 20 columns	Supplemental product, service-solution, IP, service-solution revenue, and delivery-related fields; contains repeated opportunity_id values.	Secondary supplemental source. Selected fields are collapsed to one row per opportunity_id before merging into opps2.
opps2 CRM export	8,695 rows 33 columns	Richer CRM opportunity schema with owner, status, sales stage, probability, revenue, date, duration, and proposal-related fields.	Primary base table for opportunity_df. It defines the opportunity-level grain for capacity scoring and dashboard inputs.
Proposal / response JSON	19 document pairs	Historical proposal and response text.	Used for RFP text preprocessing, chunking, embedding, retrieval, and assignment support.

Table 1: Overview of the main project data sources used in the first two weeks. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

likely reflect service-level or product-level detail rather than distinct opportunities. A direct merge from opps1 into opps2 would therefore multiply some opportunity rows and risk double-counting workload in later capacity scoring.

To preserve the intended data grain, the merge pipeline collapses opps1 supplemental fields to one row per opportunity_id before joining them to opps2. Duplicate and detail rows from opps1 are preserved in local audit outputs rather than being used to expand the final opportunity_df.

2.3 Proposal and RFP Text Data

The proposal JSON contains historical proposal and response document pairs. These documents are text-heavy and many are too long to embed as single inputs. Therefore, the RFP pipeline prepares proposal text through chunking before later embedding and vector-store integration.

The proposal text supports the RFP Assignment Tool by enabling retrieval of similar historical examples. For a new RFP, the system will retrieve related proposal chunks and combine this semantic evidence with structured opportunity features such as director experience, available capacity, service-solution fit, and risk.

2.4 EDA Findings and Design Implications

The EDA directly influenced several early design decisions. First, because opps2 has the richer CRM schema and preserves the opportunity-level grain more cleanly, it is used as the base table for opportunity_df. Second, because opps1 contains useful service and product fields but also duplicated opportunity IDs, it is treated as a sup-

plemental source rather than a second base table. Third, because the proposal documents are long, the RFP pipeline uses chunking before embedding or retrieval-style processing. Finally, because the data does not contain labeled capacity categories or gold-standard RFP assignments, the initial system will use transparent formula-based scoring and retrieval-based recommendation instead of supervised learning.

3 System Overview

The project has two core deliverables. The first is a Director Capacity Dashboard. This dashboard uses historical CRM opportunity data to estimate each director’s current workload, compare the current workload against the director’s historical baseline, and display capacity labels such as *Available*, *At Capacity*, and *Overextended*.

The dashboard also exposes drivers of the score, including open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. This is important because stakeholders need not only a final capacity label, but also an explanation of why a director appears available or overextended.

The second deliverable is an RFP Assignment Tool. For a new RFP, the tool is designed to estimate effort, retrieve similar historical proposal examples, and recommend directors based on experience, available capacity, service-solution fit, and risk. The retrieval component is designed to use chunked proposal text, embeddings, and a local vector store. In the Week 2 prototype, the RFP pipeline focuses on preprocessing, chunking, meta-data structure, and retrieval-style output before full

Merge statistic	Value
opps1 rows	8,435
opps2 rows	8,695
opps1 unique IDs	7,679
opps2 unique IDs	8,695
Matched opportunity IDs	7,628
opps1-exclusive IDs	51
opps2-only IDs	1,067
Final opportunity_df rows	8,746
Duplicate IDs in final output	0

Table 2: Week 1 opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

Azure OpenAI and Chroma integration.

The current architecture is modular. The opportunity cleaning pipeline produces a local `opportunity_df` and a cleaned downstream dataset for validation and scoring. The capacity scoring module consumes this data to compute director-level workload and capacity metrics. The Streamlit dashboard displays capacity metrics and provides an interface for RFP assignment. The RFP pipeline preprocesses proposal text, creates chunks, and supports a retrieval-oriented output schema for historical examples and future director recommendation.

4 Weekly Implementation Progress

4.1 Week 1: Data Foundation and Module Skeletons

During Week 1, the team focused on building the first reproducible data foundation and creating skeletons for the main system components. The goal was not to finalize the full capacity model, dashboard, or RFP assignment tool. Instead, we prioritized stable data structures, module boundaries, and interfaces that can support later development.

The work was divided across five roles. Yixiao focused on opportunity data merging, schema comparison, and duplicate / unmatched record summaries. Kian focused on validation of revenue, date, duration, probability, status, and sales-stage fields. Lyken focused on the capacity scoring skeleton and director table prototype. Freya focused on the Streamlit dashboard skeleton and RFP page placeholder. Jai focused on the RFP preprocessing and vector store skeleton.

For the opportunity data merge, we implemented a reproducible pipeline that loads the two anonymized CRM Excel exports, normalizes col-

umn names, compares schemas, uses `opps2` as the base opportunity-level table, and supplements selected fields from `opps1`. The pipeline uses `opportunity_id` as the required join key.

A key Week 1 finding was that `opps1` contains duplicate `opportunity_id` rows. These duplicates likely reflect service-level or product-level detail rather than separate opportunities. A direct merge from `opps1` into `opps2` created one-to-many expansion, which would risk double-counting workload in downstream capacity scoring.

To preserve the intended opportunity-level grain, we collapsed `opps1` supplemental fields to one row per `opportunity_id` before merging. Duplicate and detail rows from `opps1` are preserved in local audit outputs instead of multiplying the final `opportunity_df`. The final local `opportunity_df` contains 8,746 rows, matching the expected no-expansion count of 8,695 `opps2` base rows plus 51 `opps1`-exclusive opportunity IDs. The final merged dataset contains zero duplicate `opportunity_id` rows.

All generated processed data files are kept local and are not committed to GitHub because they are derived from sensitive CGI data. The reproducible code and tests are committed instead, so other team members with local access to the source files can regenerate the same outputs.

4.2 Week 2: From Skeletons to First Integrated Prototype

In Week 2, the team moved from separate data and module skeletons toward a first integrated prototype. The main development path connected the cleaned opportunity data to validation, owner-level capacity scoring, and Streamlit dashboard display. In parallel, the RFP pipeline progressed from placeholder functions toward sample text preprocessing, chunking, and retrieval-style output.

Yixiao refined the Week 1 merge output into a cleaner downstream dataset for validation, scoring, and dashboard development. This work focused on stabilizing `cleaned_opportunity_df`, confirming the required columns for downstream modules, preserving source and quality flags, and preparing an owner-level base summary using `opportunity_owner`. The goal was not to redesign the Week 1 merge logic, but to make the merged data easier for Kian’s validation work, Lyken’s scoring module, and Freya’s dashboard integration to consume.

Kian extended the validation work from ba-

Module	Owner	Week 2 update	Main output
Data integration	Yixiao	Refined the Week 1 merge output into a downstream-ready cleaned dataset.	cleaned_opportunity_df, column dictionary, quality flags, and owner-level summary.
Data validation	Kian	Validated fields needed for capacity scoring and dashboard use.	Missingness summary, reliable / risky fields, revenue fallback hierarchy, and date / duration assumptions.
Capacity scoring	Lyken	Implemented the first working owner-level capacity scoring workflow.	director_capacity_df, relative load, capacity score, capacity label, and scoring sanity checks.
Dashboard	Freya	Connected the dashboard to real or schema-aligned scoring output.	Capacity score chart, owner-level table, filters, and explanation placeholders.
RFP pipeline	Jai	Built an independently runnable RFP preprocessing and retrieval-style prototype.	Sample chunks, mock or basic retrieval output, and assignment_context schema.

Table 3: Week 2 progress from separate skeletons toward the first integrated prototype.

sic field inspection to scoring-oriented field assessment. The validation process reviewed revenue fields, date and duration fields, probability, status, sales stage, opportunity_owner, and opportunity_manager. The output separates fields that are reliable, fields that are usable with fallback assumptions, and fields that remain risky for formal scoring. This validation layer is important because the capacity model depends on transparent assumptions rather than supervised labels.

Lyken converted the Week 1 capacity_engine.py skeleton into a first working capacity scoring workflow. The scoring module computes sales load, inferred delivery load, current owner-level load, historical baseline, relative load, bounded capacity score, and capacity labels. The resulting director-level table provides the first version of director_capacity_df, including fields such as current_total_load, historical_baseline, relative_load_owner, capacity_score_owner, and capacity_label.

Freya updated the Streamlit dashboard from a mock-data skeleton toward an integrated capacity dashboard. The Week 2 dashboard reads either the first director_capacity_df output or a schema-aligned sample output with the same expected columns. The dashboard displays capacity score by director, capacity labels, current load versus historical baseline, and an owner-level capacity table. Filters and explanation placeholders were also retained so that the interface can continue to evolve as scoring assumptions are refined.

Jai advanced the RFP pipeline from skeleton files toward an independently runnable prototype. The Week 2 work focused on RFP text preprocessing,

chunk generation, chunk metadata structure, and a stable assignment_context output schema for dashboard integration. The prototype can produce sample chunks and basic or mock retrieval-style output. Full Azure OpenAI and Chroma integration remain later steps, but the Week 2 interface gives the dashboard a stable structure for retrieved examples, recommended directors, risk flags, and explanatory notes.

Overall, Week 2 established the first end-to-end project workflow: cleaned opportunity data supports validation; validation findings inform capacity scoring; capacity scoring produces a dashboard-ready director table; and the dashboard begins displaying capacity outputs. The RFP pipeline remains partly independent from the capacity workflow, but it now has a clearer preprocessing and output contract for future recommendation integration.

5 Current Methodology and Prototype Design

The current capacity model is transparent and formula-based. Current workload combines sales pipeline load and inferred delivery load. Sales pipeline load uses CRM fields such as sales stage, probability, estimated revenue, and project duration. Delivery load uses won opportunities and date or duration fields as a proxy for active delivery work. This approach keeps the scoring logic interpretable and avoids using an LLM as the source of truth for numeric capacity scores.

Each director’s current load is compared against their own historical baseline. This is important because different directors may normally

carry different amounts of work. The prototype keeps both `capacity_score_owner` and `relative_load_owner`. The capacity score provides a bounded headroom metric for the main dashboard display, while relative load shows how far a director is above or below their own historical baseline. In the Week 2 prototype, these fields are produced in a director-level capacity table and then renamed where needed for dashboard display.

The Week 2 scoring workflow uses the cleaned opportunity data and validation findings as inputs. Revenue, date, duration, probability, status, sales stage, and owner fields are checked before being used in scoring. Fields that are reliable can be used directly, while fields with known data quality limitations require fallback assumptions. This validation-driven design is important because the project does not have ground-truth labels for director capacity, so scoring must remain explainable and auditable.

The Streamlit dashboard consumes the director-level capacity output rather than recomputing the scoring logic inside the interface. The dashboard displays capacity score by director, capacity labels, current load versus historical baseline, and an owner-level table. This separation keeps the scoring layer and visualization layer modular: the capacity model can be adjusted without redesigning the dashboard, and the dashboard can be tested with either real scoring output or schema-aligned sample data.

For RFP assignment, the current prototype combines proposal text preprocessing with a retrieval-oriented output structure. Historical proposals are prepared for chunking, and the pipeline now supports sample chunks, chunk metadata, and a stable `assignment_context` schema for dashboard integration. The intended full approach remains to embed historical proposal chunks and store them in a local vector store, then retrieve similar historical examples for a new RFP. In the Week 2 prototype, the emphasis is on making the preprocessing, chunking, retrieval interface, and dashboard output contract stable before final Azure OpenAI and Chroma integration.

The planned recommendation logic combines semantic relevance with structured opportunity features. Similar historical proposal evidence will be combined with director experience, available capacity, service-solution fit, and risk indicators. Experience remains the highest-priority signal, followed by capacity and service-solution fit. At the

current stage, the RFP pipeline is treated as a prototype interface rather than a final recommendation engine.

6 Project Management and Next Steps

In Week 2, the team continued the five-role structure established in Week 1, but shifted the coordination focus from separate module setup to interface alignment and early integration. The main capacity workflow now depends on a clearer handoff sequence: the cleaned opportunity data supports validation; validation findings inform the scoring assumptions; the scoring module produces a director-level capacity table; and the dashboard consumes that table for visualization.

This coordination structure helped reduce blocking between modules. Yixiao's cleaned opportunity data and column dictionary gave Kian a stable input for field validation. Kian's validation findings and fallback assumptions informed Lyken's scoring implementation. Lyken's dashboard-ready capacity output allowed Freya to move the Streamlit dashboard beyond pure mock data. In parallel, Jai kept the RFP pipeline prototype relatively independent by using sample proposal text, mock director data where needed, and a stable `assignment_context` schema for future dashboard integration.

The next stage of development will focus on testing and refinement rather than only interface construction. In Week 3, the team will validate the capacity scoring output, check whether the resulting director labels are reasonable, refine threshold choices, and continue connecting the dashboard to real scoring outputs. The dashboard will also need additional usability checks, including filter behavior, score explanations, and handling of missing or sparse data. For the RFP tool, the next steps are to improve retrieval quality, test chunking behavior on historical proposal text, and begin moving from mock or basic retrieval output toward the Azure OpenAI and Chroma-based retrieval pipeline.

Because the Director Capacity Dashboard is the first core deliverable, it remains the highest priority if the team falls behind. The RFP Assignment Tool will continue to progress in parallel, but its recommendation logic can remain simpler until the capacity scoring and dashboard workflow are stable.

7 Current Assumptions and Risks

Several assumptions and risks remain at the end of Week 2. First, `opportunity_id` is still treated as the required join key for opportunity-level merging. Second, `opps2` remains the primary base CRM opportunity table, while `opps1` is used only for selected supplemental fields after being collapsed to one row per `opportunity_id`. Third, `opportunity_owner` is treated as the main director-level entity for MVP capacity scoring, while `opportunity_manager` is retained as optional metadata.

The capacity scoring workflow also depends on data quality assumptions. Revenue, date, duration, probability, status, and sales-stage fields can support the prototype only after validation and fallback rules are applied where necessary. If these fields are incomplete or inconsistent, the resulting workload estimates may be directionally useful but should not be interpreted as exact measures of director capacity.

The Week 2 scoring thresholds and capacity labels are still preliminary. Labels such as `Available`, `At Capacity`, and `Overextended` are intended to support stakeholder review, not to serve as final business rules. The dashboard should therefore present the capacity score as a decision-support signal together with score drivers and explanations, rather than as an absolute ranking of directors.

The RFP pipeline also remains at a prototype stage. Although the Week 2 interface supports preprocessing, chunking, sample retrieval-style output, and an `assignment_context` schema, full Azure OpenAI embedding and Chroma retrieval integration remain future work. Until that integration is complete and validated, RFP recommendations should be treated as prototype outputs.

Finally, the project still lacks ground-truth labels for director capacity, RFP effort, and correct director assignment. Evaluation will therefore rely on stakeholder review, threshold sensitivity checks, sanity checks, and selected case studies rather than standard supervised-learning metrics.

8 Conclusion

This Week 2 version extends the initial project report from a data-foundation draft into a first integrated prototype report. Week 1 established the reproducible opportunity merge, preserved opportunity-level grain, and created the main mod-

ule skeletons. Week 2 refined the cleaned opportunity data, added validation-oriented field assessment, produced an initial director-level capacity scoring output, connected the dashboard to capacity results, and advanced the RFP pipeline toward preprocessing, chunking, and retrieval-style output.

The next stage will focus on validating scoring behavior, refining thresholds and labels, improving dashboard usability, and strengthening the RFP retrieval and recommendation workflow. The project remains an applied decision-support system rather than a benchmark-driven modeling task, so future evaluation will emphasize interpretability, stakeholder review, reproducibility, and practical usefulness.